

安家栋

求职意向：Golang | 深圳

31岁 | 男 | 汉族 | 贵州 | 7年经验 | 深圳市宝安区 | 闽南师范大学 | 15届计算机科学与技术专业
15260605670 | 15260605670@163.com

专业技能

- 架构判断与建模**：具备体系化的业务建模与架构决策能力——业务原型库、建模四问、角色视角法、三条黄金线拆服务；CP/AP 取舍、容量评估、缓存与消息队列选型、分布式事务选型；习惯从"业务不变式 + 物理约束"反推架构，而不是从工具堆栈正推。
- 一致性与资金安全**：多级幂等、状态机 CAS、Lua 原子结算、多维限量防超发、端到端对账等在线场景资金安全范式；跨多个高价值业务零资损落地。
- 高并发与实时数据**：主导过 1 万+ QPS 抽奖、直播连麦 PK 与实时音视频端云协同、秒级实时推荐等场景的容量设计与调优；对流量形态分层（大房间无状态路由 / 小房间一致性 Hash、大 Key 分片、异步合并写等）有工程判断，而非依赖堆机器。
- 实时流水线与多存储选型**：Flink + Kafka + Redis + ES 组合下的大规模实时管道落地经验（Keyed State 微批、EventTime + Watermark、异步 IO、双流合并去重、多 Job 时序协调）；对 MySQL / Redis / ES / 对象存储 / 向量存储的"按访问模式选存储"有清晰判断路径。
- 平台化与扩展性设计**：主流程固化 + 显式扩展点的中台建设经验，把"哪些固化、哪些放开、哪些走配置、哪些走代码"作为平台稳定性的第一红线。
- 技术栈**：主力 Go，熟悉 Java / Python；熟悉主流 RPC（gRPC / tRPC）与服务治理生态；熟悉 TCP / WebSocket / RTMP / WebRTC 等协议，具备音视频端云协同经验；了解湖仓一体、CDC 同步、向量检索等大数据与 AI 工程组件。
- 方法论沉淀**：输出两份个人手册（《架构设计方法论》/《后端技术演进全景》），覆盖从业务建模、系统架构到技术演进脉络的完整判断链路。

工作经历

2023-12 ~ 至今

腾讯游戏

后端开发工程师

项目：互娱通用抽奖系统（核心开发者）

项目背景：深度参与腾讯游戏旗下通用抽奖系统的架构演进与核心优化。系统通过插件化架构被王者荣耀、英雄联盟、剑灵、剑侠情缘等 10+ 款游戏复用，支撑经典 Gacha、集卡合成、自选许愿池、无放回盲盒等各类商业化抽奖活动。本人作为核心开发者负责多个核心模块的重构优化与新游戏接入。

技术特征：插件化架构支撑 10+ 款游戏复用，每个活动独立部署、支持水平扩展；核心主流程固化 + 10+ Hook 扩展点，新游戏接入周期从 2 周缩短至 2 天；三级幂等 + 四层限量 + Layer 多轮隔离保障资金安全；某头部游戏春节抽奖活动实际峰值 1 万+ QPS（付费抽奖场景，转化链路长），抽奖接口 P99 < 50ms，限量道具零超发。

主要工作内容：

- 架构判断**：把"为某游戏定制"演进为"平台化复用"。识别抽奖领域的核心业务原型（概率组、加权采样、限量、幂等、状态机），将主流程固化（参数校验→概率组确定→多维过滤→加权采样→限量检查→状态持久化→响应缓存），抽象出 10+ 个 Hook 点 + "后注册插件覆盖先注册实现"的显式扩展机制。本人负责多个核心插件的开发与优化，并参与多款新游戏接入适配（如剑侠情缘按服务器等级选择奖池、LOL 定制道具响应格式等），通过差异化插件将新游戏接入周期从 2 周缩短至 2 天。**设计取舍**：早期讨论过"配置驱动一切"的方案，但游戏侧的差异化诉求常常越过通用配置的表达边界；最终坚持"主流程固化 + 显式扩展点"，把"哪些固化、哪些放开"作为平台化的第一红线。
- 优化分层概率模型与采样机制**：核心权衡是「per-user / per-request 概率分布动态变化」vs「采样性能」——预处理别名采样虽 O(1) 但每次过滤都要重建表，得不偿失，故选择轮盘赌加权随机（O(n) 但零预处理，n 通常 < 500 可接受）。Fixed / Unfixed 双轨缩放机制保证 Fixed 概率不受过滤干扰、Unfixed 按比例自动重分配，业务侧仅需配置无需理解算法；扩展支持概率组嵌套（最大 10 层递归）。随机数采用 CSPRNG 安全种子 + xoshiro256 快速 PRNG 组合，兼顾不可预测性与吞吐。
- 重构并完善三级幂等保障机制**：起因是一次线上事故——中间链路异常导致迟到请求覆盖了已经写入的缓存，险些资损。复盘后梳理并加固三级幂等体系：第一层响应缓存幂等（命中则直接返回，在加锁前执行以减少 Redis 竞争）；第二层状态回退 / 回放（双缓存模式回退 PrevState 重新随机，或按 PlaybackReward 逐轮重放历史结果）；第三层 Redis 分布式锁（SETNX 防并发抽奖）。与下游发货组件的唯一索引配合，实现端到端幂等。事后把"三级幂等是否闭合"作为新场景接入评审的强制 checklist——**边界不闭合往往比组件出 bug 更危险**。
- 优化四层限量管控机制**：系统支持四层限量维度（用户累计 / 用户日 / 全服日期 / 全服永久），通过 Redis 原子递增计数实现并发安全——采样命中限量道具后原子递增计数器，超限则触发重抽，重抽时 Filter 提前过滤。**关键不变式**：必须保证"已发放数 ≤ 上限"在任何时刻成立，所以加计数原子操作必须早于状态写入，否则两者之间的窗口就会超发。这条不变式写进了设计文档作为后续维护的红线。
- 扩展里程碑保底系统**：在基础幸运值机制上扩展支持多规则多优先级配置，通过幸运值累加 + 阈值切换概率组，覆盖软保底（概率递增）、

- 大小保底、跨池继承等业界全部主流玩法。**配置 vs 代码的取舍**：保底玩法运营频繁出新花样，做成配置驱动让运营自助迭代；限量、幂等关系到资损，做成代码强约束不允许配置绕过——把"资金安全边界"与"运营创造力空间"显式分层。
- **设计多层抽奖隔离机制**：设计 Layer 机制实现多轮次抽奖的完整隔离——每个 Layer 拥有独立的 Redis 用户状态、奖池道具集合、概率表和响应缓存，切换 Layer 等于开启全新一局。支持道具按 Layer 归属配置（精确匹配 / 通配 / 周期匹配），初始化时按 Layer 独立构建奖池。多轮活动（抽完切换下一轮）、分段抽奖（不同 Layer 不同概率）、赛季制（赛季重置切新 Layer）等业务需求看似不同，但都映射到同一套 Layer 隔离模型上，运营改配置即可，不需要改代码。
 - **作为中台开发接口人跟进多款已接入游戏**：作为中台侧的开发接口人对接已接入游戏的后续迭代——把游戏侧的差异化诉求翻译为"插件清单"或参数变更，与游戏方明确"哪些差异由中台承接、哪些由业务侧自己实现"以避免责任真空；版本上线前与游戏侧共同梳理已知风险（春节大流量、首充节点、礼包叠加等），按"白名单 → 1% → 10% → 全量"灰度推进。**核心原则**：中台稳定性优先级高于单一业务的灵活度，遇到"绕过主流程做特例"的诉求会先尝试抽象为通用扩展点，不能抽象就拒绝——中台的价值不是"什么都能做"，而是"边界清晰、约束可信"。

项目：王者新游推荐系统（协助方 / 数据底层与服务侧）

项目背景：WeDo 是面向微信/QQ 生态的游戏社交平台，覆盖车队（临时组队）、战队（长期社交）、对局回放三大推荐场景。本人**作为协助方**承担推荐系统的**实时数据底层（Flink）与推荐服务（Go/tRPC）建设**，与算法、业务、推荐主导团队协同推进，实时好友关系链数据在玩家登录后秒级可用。

技术特征：Flink 多 Job 流水线承接登录流、变更流、屏蔽流、对局流等多源异构事件，通过 Keyed State + EventTime + Watermark 实现乱序容忍与幂等 upsert；关系链数据入湖到 Redis + ES 双存储，读写分离分别承接推荐召回与详情查询；推荐服务以 DAG 编排算法节点、配置热更新，命中缓存耗时 <50ms、全链路 P99 < 1000ms；对局回放索引采用多 Job 分阶段拼装 + 防抖 Timer 协调下游就绪时序，配合 Redis 预热将高频随机读压力从 ES 卸载。

主要工作内容：

- **好友关系链实时管道**（车队 / 战队推荐共用底层数据源）：基于 Flink 双 Kafka 主从容灾消费，全量登录流通过 Keyed State 微批聚合 + 异步 IO 读取 Redis 历史数据，实现日志与历史的三路合并去重；增量变更流按事件类型精细分发，并对跨平台好友关系做删除保护，玩家登录后好友数据秒级可见。与上游团队对齐了"事件分主题输出 + 我侧承诺端到端 P99 < 5s + 至少一次语义"的接口约定，明确各自的容错边界——**接口先行胜过事后追责**。
- **车队 / 战队 ES 实时同步**：通过 KeyedProcessFunction + Timer 在短窗口内合并高频字段变更，结合 UpdateTime 乱序保护避免老流水覆盖新状态，按主键 upsert 保障幂等；战队场景额外融合登录 / 登出 / 成员进出四路事件流，以 EventTime 窗口 + Watermark 容忍乱序，实时统计每支战队在线成员数。
- **战队推荐屏蔽列表增量维护**：基于时间窗口聚合用户的屏蔽 / 取消 / 清空操作，处理"清空后再屏蔽"的时序冲突；与 Redis 现有列表异步合并后写回，为推荐召回提供过滤数据。
- **Go 推荐服务**：以 DAG 方式编排算法节点，算法流程通过配置热更新；融合 ES 车队详情与好友关系链，按好友数 / 亲密度 / 组队时间 / 成员数四维排序；引入用户级分页缓存与批量并发拉取头像，命中缓存耗时 <50ms，全链路 P99 < 1000ms。
- **对局推荐多 Job 流水线**：对局回放 ES 索引由多个 Flink Job 分阶段 upsert 拼装，承担其中数据接入与中转 Job 的设计实现，通过多标志位完整性校验 + 防抖 Timer（3s/4s/8s）协调下游 Job 数据就绪时序；设计中转 Job 将 ES 完整文档预热到 Redis，将下游高频随机读压力从 ES 转移到 Redis。在"单 Job 大而全"与"多 Job 拆分"的方案讨论中倾向后者——上游数据源迭代节奏不同（三条黄金线中的"变化频率线"），单 Job 会让所有改动被最慢一方拖慢，分 Job 后各自独立演进，代价是多了几个标志位协调的逻辑。

2020-07 ~ 2023-09腾讯音乐后台开发工程师

项目：QQ 音乐直播系统（核心开发者）

项目背景：参与 QQ 音乐直播后台系统的核心功能建设，基于腾讯 TRTC 实时音视频服务构建，支撑开播/关播、进房/退房、连麦/PK、弹幕消息、房间管理、在线观众管理等全链路功能。本人**作为核心开发者**主要负责连麦/PK 的设计与开发、直播弹幕系统的部署维护，同时参与开播、进退房、在线观众管理等模块。

技术特征：基于腾讯 TRTC 实时音视频服务构建，连麦端到端延迟 < 300ms；房间、连麦、弹幕等模块解耦独立部署，支持水平扩展；关键状态与信令依托 Redis + 长连接 IM 组件，保障连麦 PK、麦位、结算等场景的一致性与可靠性。

主要工作内容：

- **负责连麦 / PK 状态机设计与实现**：设计完整的 PK 生命周期状态机（空闲→邀请中(匹配中)→连接中→PK 中→结算中→空闲），每次状态变更携带版本号做 CAS 校验防并发跳变。**核心难点是跨房 PK**：两个主播分属不同 TRTC 房间，需通过信令协调双方进行 RTC 跨房连接，再由 MCU 混流旁路推送至各自 CDN 流，普通观众无感知切换。设计了邀请超时取消（30s）、拒绝 / 无响应回退、断线保护（15s 超时自动判

- 负结算) 等容错策略。建模时把双方主播的状态机用共享版本号绑定, 邀请、匹配、断线等异常分支都作为这套核心模型的边界态处理, 避免每个异常各写一套逻辑——**用一个强不变式吸收所有异常分支**。
- 负责 PK 积分系统与实时结算机制: 积分由支付 / 互动服务回调驱动, 采用 Redis Lua 脚本原子执行去重检查 + 总积分递增 + 贡献榜递增 (ZINCRBY), 解决礼物重复计分问题。设计了基于 Redis ZSet + 500ms 轮询的倒计时管理; 结算通过 Lua 脚本原子读取双方积分并写入结果, 同时置状态为 finished; 加分脚本检测该标志位实现结算与加分的逻辑互斥, 避免最后一刻加分改变已判定胜负——**结算后积分快照不可被修改**是这套机制的核心不变式。
 - 负责麦位管理与连麦信令可靠性保障: 设计麦位席位状态管理 (空闲→申请中→已上麦→空闲 / 已锁定)。上麦流程: 用户申请→主播审批→服务端签发 TRTC UserSig→用户进房推流→收到 TRTC 进房回调后更新席位并触发混流布局重算→广播变更事件。**关键设计**: 上麦以服务端收到 TRTC 回调为最终确认而非客户端自报; 关键信令通过长连接下发并要求 ACK, 超时重试防弱网丢失——把"最终事实"锚定在权威源, 而不是任何单侧客户端的自报。
 - 参与开关播流程与并发控制: 实现 CreateShow 完整流程 (前置检查→生成 ShowInfo→后置处理), 包含权限 / 实名 / 重复开播校验、全局唯一 ShowID 生成、流地址签发等。通过 Lua 脚本保证 3s 内同一用户创建的 ShowInfo 幂等; 跨设备重复开播通过 UDID 判别——同设备恢复直播, 不同设备拒绝并提示。
 - 参与进退房流程与在线观众管理: 进房获取 TRTC 密钥 / IM 命令字 / CDN 流地址, 根据房间规模自动切换 TRTC 进房或 CDN 进房 (TRTC 上限 10 万人)。设计快速进房优化: 并行执行 TRTC 进房与业务进房, 用户先看到画面再加载装扮信息。在线观众通过消息队列解耦上报至在线观众组件, 客户端 30s 心跳维护在线状态; 排行榜按 Key 聚合批量并行计算提升吞吐量, 对账进程定期校验数据准确性——**在线态用最终一致, 账务数据靠对账兜底**。
 - 负责直播弹幕系统的部署维护与容量调优: 基于团队 IM 弹幕组件, 系统采用长连接 + 长轮询拉模型 (200ms 同步间隔, 时延 < 1s)。针对**不同流量形态采用不同策略**: 大房间无状态路由、小房间一致性 Hash; 消息按优先级隔离存储 (大喇叭→信令→礼物→普通群消息), 保证关键信令不被普通弹幕淹没——**按流量形态拆策略, 而不是靠加机器**。

2018-07 ~ 2020-06

博雅互动

实习/后端开发工程师

主要工作:

- 参与"德州扑克"微服务改造: 将单体服务拆分为微服务架构, 负责牌局统计服务、比赛服务、赛事管理服务模块的拆分与独立部署。
- 参与 266 项目开发: 负责任务体系的设计与开发 (任务领取、进度追踪、奖励发放), 以及活动中心服务的重构。

个人亮点

- 能从模糊业务推导出技术方案: 习惯先按"建模四问" (名词 / 动作 / 查询 / 约束) 把业务拆成实体、事件、读模型、一致性边界, 再决定技术选型; 对 CAP / BASE 取舍、缓存一致性、分布式事务选型、容量评估有体系化的推导路径, 而不是凭经验拍脑袋。
- 平台化复用经验: 抽奖系统通过插件化让 10+ 款游戏共用一套核心, 新游戏接入从 2 周缩短到 2 天; 过程中踩过"配置驱动一切"的坑, 也吃过"主流程被改穿"的亏——对"哪些固化、哪些放开"、"哪些走中台、哪些留给业务"有落地过的判断, 而不是纸面结论。
- 一致性与防超发设计: 三级幂等 + 四层限量、Lua 原子结算与加分互斥、跨房 PK 状态机 CAS——多个在线场景零资损; 做过线上幂等失效问题从定位到修复的完整闭环, 深刻理解"**边界不闭合比"组件出 bug"更危险**"这条工程铁律。
- 实时数据与高并发系统调优: Flink 多 Job 流水线、ES 实时同步、弹幕大小房间隔离、连麦端云协同——做过的调优更多是"按业务流量形态拆策略"而不是"加机器", 对什么时候该缓存、该限流、该合并写、该异步化有具体的判断。
- 跨团队配合: 在抽奖系统中作为中台开发接口人对接多款游戏的后续迭代, 在直播 PK 中作为 Leader 牵头下的核心执行者参与多方契约对齐——熟悉中台和业务方各自的视角与边界, 知道**接口边界怎么定、异常责任怎么划、灰度怎么推**。

技术视野 · 智驾数据闭环

主动学习自动驾驶技术演进与数据闭环工程实践, 尝试把互联网侧的高并发、实时数据、多存储选型经验对应到智驾工程域; 目前尚无智驾方向实际项目经验, 以下为学习整理后的理解框架。

- 对智驾技术演进的整体理解: 智驾从"规则驱动 → 模块化 ADAS → CNN 感知 → BEV + Transformer → 端到端 → VLA / 世界模型"完成了六次范式跃迁。2023 年之后行业的核心竞争维度已经从"拼算法"迁移到"拼数据闭环转速"——**谁能把长尾场景更快地变成训练样本, 谁就赢**。当前头部厂商普遍采用"模块化保底栈 + 端到端上限栈 + 世界模型辅助"的组合范式, 而不是纯端到端叙事。
- 对行业战略框架的理解 (一个飞轮 + 两条腿): 智驾竞争的完整视角不止是"数据闭环"这一台发动机——头部玩家普遍围绕**一个飞轮 + 两条腿**的组合展开:
 - 飞轮(数据闭环)**: 把长尾场景不断转化为模型能力的核心引擎, 其"闭环转速 T"是可量化的战略指标 (特斯拉~天级 / 头部中厂~周级 / 二线~月级)。
 - 腿一(规模化量产)**: 装机量既是数据的来源、也是现金流的支撑, 需要通过"旗舰 + 主销 + 普惠"三层产品矩阵与 BOM 成本工程支撑起飞轮的燃料。
 - 腿二(前沿技术制高点)**: VLA / 世界模型 / L4 Robotaxi / 移动物理 AI 等下一代范式, 决定公司在下一次范式跃迁时是引领者还是跟随

- 者，也是资本市场的估值锚。
- 三者互为因果：**只有飞轮没有量产腿会"数据饥饿"，只有量产腿没有飞轮会"长尾追不完"，只有飞轮没有前沿腿会"在下一次范式跃迁被淘汰"。作为后端工程师，我关注的是**如何让飞轮转得更快**（数据闭环工程），但也理解这个飞轮在整个战略盘面中的位置。
- **对数据闭环全景的理解：**完整闭环是一条跨端-边-云的大规模数据流水线，可拆为八环——**触发（影子模式 / 事件触发）→ 采集上传（分片断传 / 家充窗口）→ 入湖 + 元数据（对象存储 + 数据版本）→ 挖掘（规则 / 向量 / VLM 三代）→ 标注（4D 自动化）→ 训练（加权采样 / 长尾升权）→ 仿真验证（回灌 + 世界模型场景生成）→ 灰度部署（分阶段 OTA + 影子监控 + 秒级回滚）**。三个转速指标（T1 数据→样本 < 24h、T2 样本→模型 < 7 天、T3 模型→车端 < 14 天）构成第一梯队的工程指标线。
 - **对后端工程侧挑战的映射：**智驾数据闭环的很多痛点，本质是我在互联网侧处理过的问题在新领域的变体——
 - 海量数据存储与检索：**对应对象存储 + 数据湖（Iceberg）+ 多存储分工（MySQL 元数据 / ClickHouse 分析 / ES 全文 / Milvus 向量）——延续在游戏 / 推荐系统里"按访问模式选存储"的判断路径。
 - 长尾场景挖掘：**从规则挖掘 → CLIP / Qwen-VL 语义向量检索 → VLM 主动挖掘，本质是**"用哪种索引匹配哪种查询语义"**——与推荐系统里 ES 多条件过滤 + 向量召回是同一类问题。
 - 数据版本管理与可复现：**每次训练必须锁定数据集 version、支持回滚到任意历史标签集——对应互联网侧的"数据即代码"（DVC / LakeFS + Iceberg Snapshot），与我在抽奖系统里坚持的"配置版本化 + 灰度回滚"是同一种工程审美。
 - 合规与脱敏：**端侧强制脱敏、境内闭环——与直播 / 游戏侧的实名合规、地域数据隔离在架构上一脉相承。
 - 训练供给性能：**WebDataset + Alluxio + K8s 混部解决 IOPS 瓶颈——与我在 Flink 侧对 Kafka / Redis / ES 的吞吐调优是同一类问题（按物理约束反推架构）。
 - **可迁移的方法论视角：**智驾数据闭环本质是"事件流 + 状态机 + 多级存储 + 幂等 pipeline"的组合——车端触发是事件源、影子模式是"读写分流的旁路验证"、4D 自动标注是"批处理原型 + 人工兜底"、灰度 OTA 是"分层放量 + 秒级回滚"。这些原型在我做过的抽奖平台、直播 PK、推荐管道里都出现过，只是承载的业务语义换成了 clip / scene / label。
 - **关键词语与视野：**了解 corner case 挖掘、shadow mode、data flywheel、场景库、真值系统、Occupancy Network、BEV、Sim2Real、SOTIF、ODD、L2/L3/L4 分级、影子模式、闭环转速、4D 自动标注 等核心概念的定义与作用，具备与领域同学进行跨领域对话的基础语言。

技术沉淀与方法论

- 《**架构设计方法论**》：上半场业务建模 + 下半场系统架构。包含 20+ 业务原型库、角色视角法、建模四问、三条黄金线拆服务；CAP / BASE 取舍、容量评估六步公式、缓存四模式与三大经典问题（穿透 / 击穿 / 雪崩）、消息队列三铁律（幂等 / 顺序 / 兜底）、分布式事务选型（本地消息表 / TCC / SAGA）、CQRS 与事件溯源等。核心信条：**在物理约束与业务 SLA 的夹缝中做最优权衡，用有限资源对抗无限复杂度。**
- 《**后端技术演进全景**》：从单机到 AI 原生 6 个纪元的演进脉络。三条主线贯穿——理论主线（ACID → CAP → BASE → FLP → Paxos / Raft）、架构主线（单体 → C/S → SOA → 微服务 → 云原生 → AI 原生）、数据主线（文件 → 关系数据库 → 分库分表 → NoSQL → NewSQL → 向量数据库）。目的是搞清楚每一项技术为什么会出现、解决了什么矛盾——**知道技术从哪里来，才能判断它要到哪里去。**